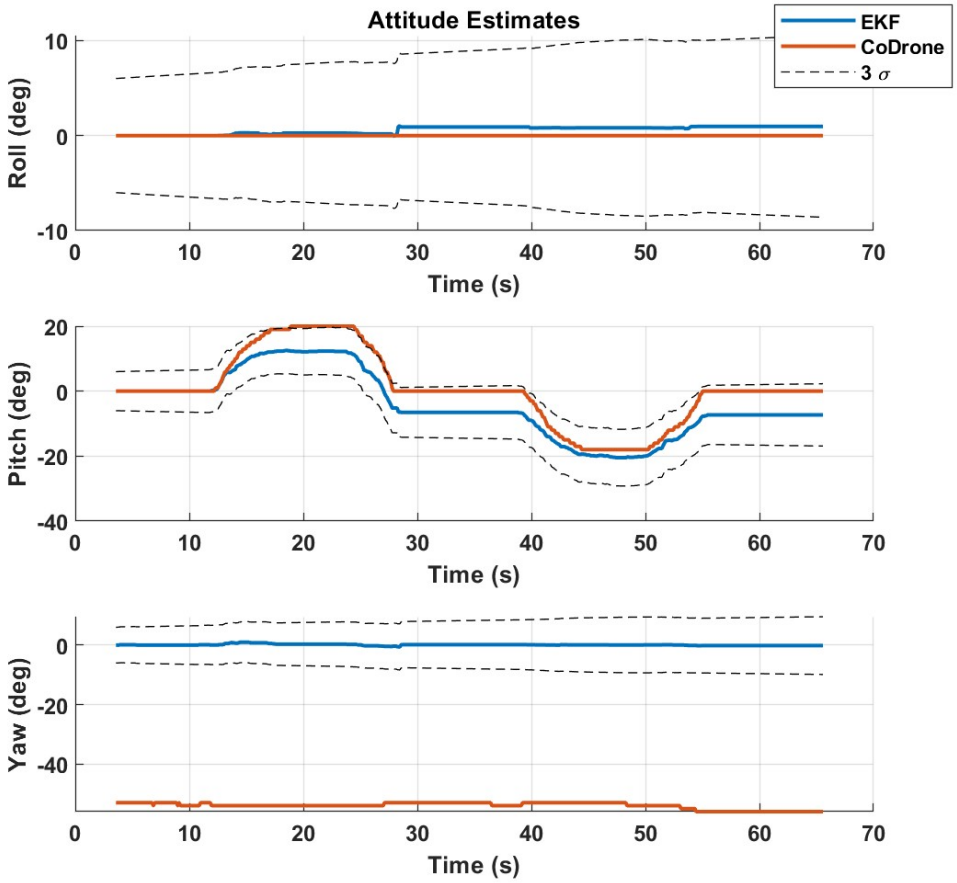
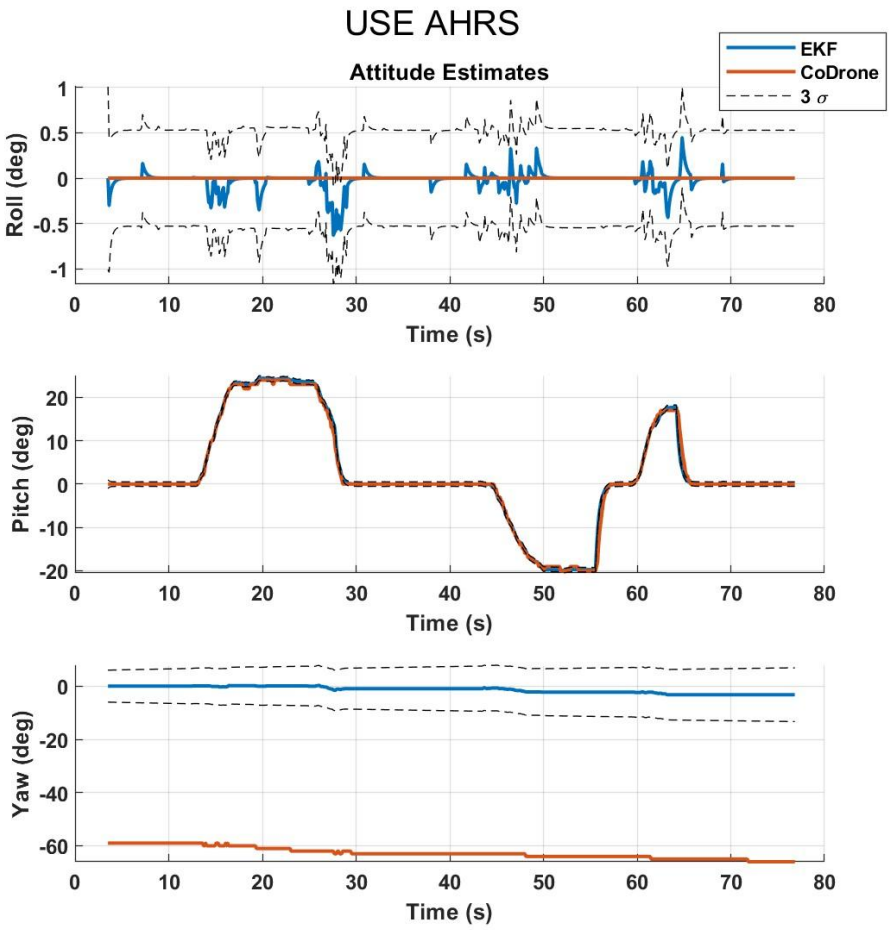
CoDrone Design Project Dimitri Medina

Part 1 – Attitude Estimate:





I would say that the performance of the pitch estimator did really well. There is very little error between the truth and estimate once the AHRS was activated. The performance of the roll had some issues, but that could just be noise during the manuever which is ok. The yaw estimate was the “worst”, when the bias was removed it followed quite closely. The error seemed to increase as time went on for USE_AHRS = FALSE, however it followed quite close to the truth with USE_AHRS = TRUE.

Part 2 – Altitude and Velocity Aiding:

$$y_{alt} = alt_{meas}$$

$$h_{alt} = -\hat{p}_z^I$$

$$C_{alt} = \begin{bmatrix} \frac{\partial h_{alt}}{\partial \phi} & \frac{\partial h_{alt}}{\partial \theta} & \frac{\partial h_{alt}}{\partial \psi} & \frac{\partial h_{alt}}{\partial p_x^I} & \frac{\partial h_{alt}}{\partial p_y^I} & \frac{\partial h_{alt}}{\partial p_z^I} & \frac{\partial h_{alt}}{\partial v_x^I} & \frac{\partial h_{alt}}{\partial v_y^I} & \frac{\partial h_{alt}}{\partial v_z^I} \end{bmatrix}$$

$$C_{alt} = [0 \quad 0 \quad 0 \quad 0 \quad 0 \quad -1 \quad 0 \quad 0 \quad 0]$$

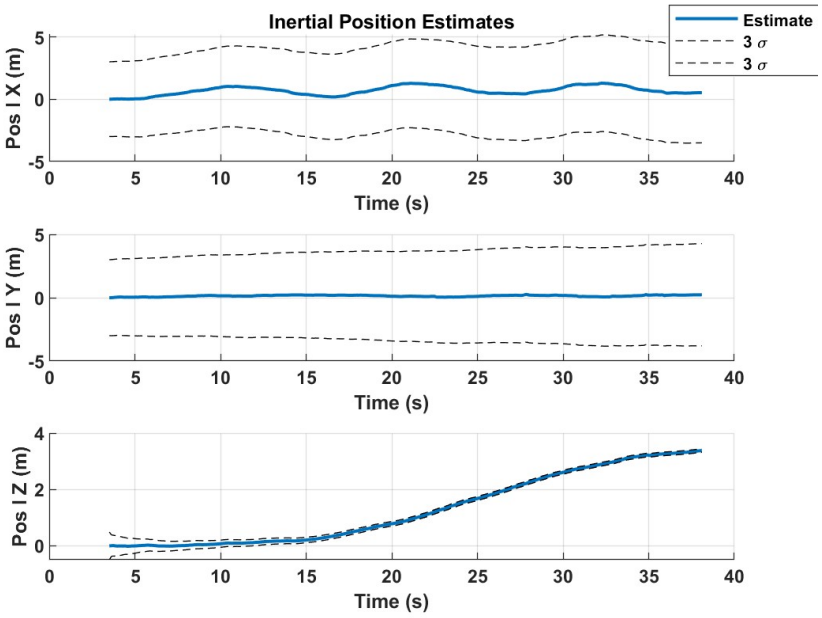
Just to note: I swapped the signs in the implementation. For some reason it worked this way. And followed the estimate a lot better.

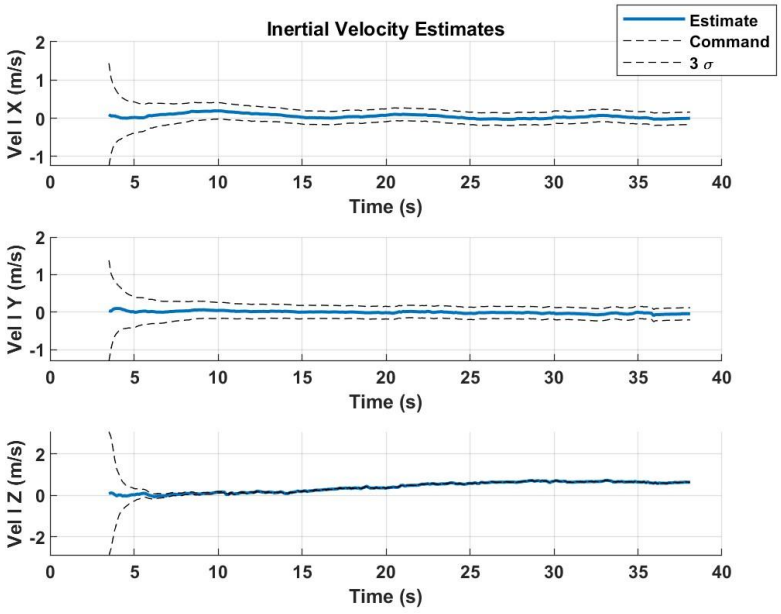
$$y_{vel} = \begin{bmatrix} v_{x_{meas}} \\ v_{y_{meas}} \end{bmatrix}$$

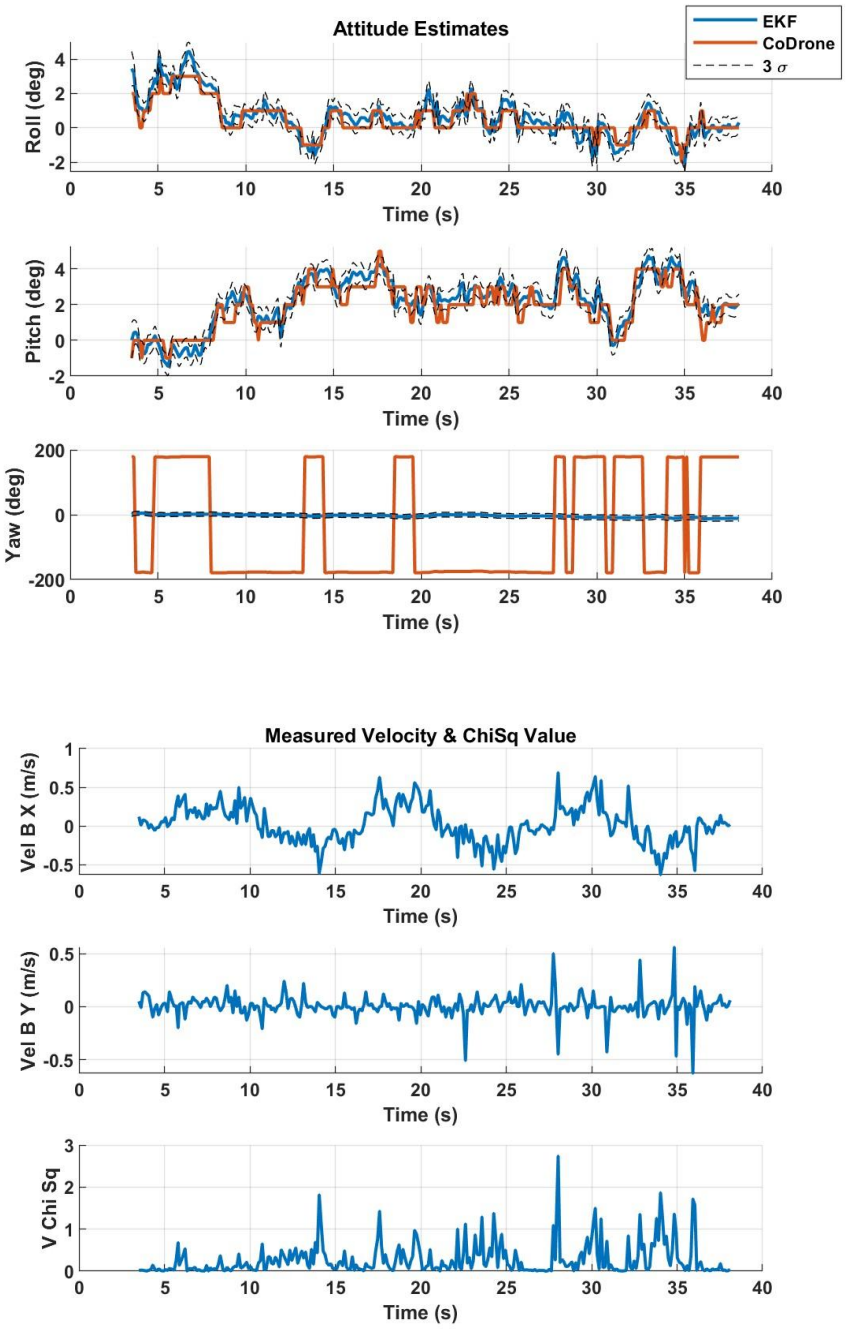
$$h_{vel} = \begin{bmatrix} \cos(\hat{\psi})\,\hat{v}_x^I + \sin(\hat{\psi})\,\hat{v}_y^I \\ -\sin(\hat{\psi})\,\hat{v}_x^I + \cos(\hat{\psi})\,\hat{v}_y^I \end{bmatrix}$$

$$C_{vel} = \begin{bmatrix} \frac{\partial h_{vel_1}}{\partial \phi} & \frac{\partial h_{vel_1}}{\partial \theta} & \frac{\partial h_{vel_1}}{\partial \psi} & \frac{\partial h_{vel_1}}{\partial p_x^I} & \frac{\partial h_{vel_1}}{\partial p_y^I} & \frac{\partial h_{vel_1}}{\partial p_z^I} & \frac{\partial h_{vel_1}}{\partial v_x^I} & \frac{\partial h_{vel_1}}{\partial v_y^I} & \frac{\partial h_{vel_1}}{\partial v_z^I} \\ \frac{\partial h_{vel_2}}{\partial \phi} & \frac{\partial h_{vel_2}}{\partial \theta} & \frac{\partial h_{vel_2}}{\partial \psi} & \frac{\partial h_{vel_2}}{\partial p_x^I} & \frac{\partial h_{vel_2}}{\partial p_y^I} & \frac{\partial h_{vel_2}}{\partial p_z^I} & \frac{\partial h_{vel_2}}{\partial v_x^I} & \frac{\partial h_{vel_2}}{\partial v_y^I} & \frac{\partial h_{vel_2}}{\partial v_z^I} \end{bmatrix}$$

$$C_{vel} = \begin{bmatrix} 0 & 0 & -\sin(\hat{\psi})\,\hat{v}_x^I + \cos(\hat{\psi})\,\hat{v}_y^I & 0 & 0 & 0 & \cos(\hat{\psi}) & \sin(\hat{\psi}) & 0 \\ 0 & 0 & -\cos(\hat{\psi})\,\hat{v}_x^I - \sin(\hat{\psi})\,\hat{v}_y^I & 0 & 0 & 0 & -\sin(\hat{\psi}) & \cos(\hat{\psi}) & 0 \end{bmatrix}$$


Inertial Position Estimates
Estimate
3 σ
3 σ
Pos I X (m)
Pos I Y (m)
Pos I Z (m)
Time (s)


Inertial Velocity Estimates
Estimate
Command
3 σ
Vel I X (m/s)
Vel I Y (m/s)
Vel I Z (m/s)
Time (s)



The position estimates for x and y are reasonable, as I only moved along the x axis with minimal movement along the y. This is also reflected in the velocity, which is reasonable along the x and y axis. The most unreasonable would have to be the Z position, which despite no movement along the Z axis, showed a rise in altitude. This could be because the

P matrix for the Z position was too small, however scaling it seemed to not work as much either.

The uncertainty in the X/Y position is so different from Z because they are derived from different measurements. Z position is derived from the barometric pressure measurements, while x and y are from the optical flow sensor. The only anomalous behavior I saw was in the Vel Z graph, showing a steady rise when no rise was occurring.

Part 3 – Position Control:

$$k_{p_{IX}} = 20$$

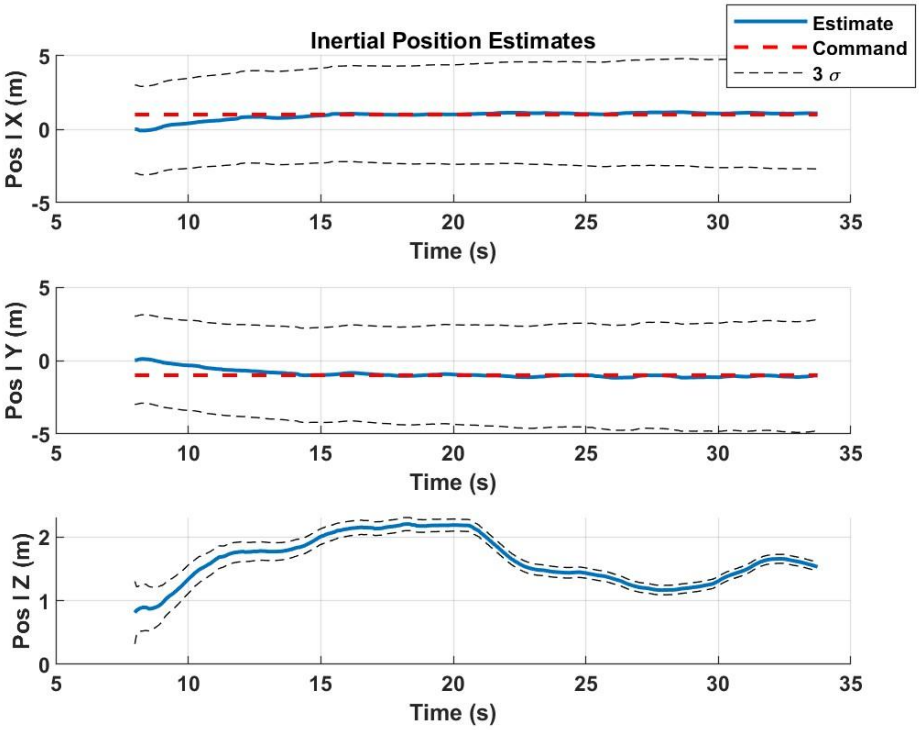
$$k_{i_{pIX}} = 4$$

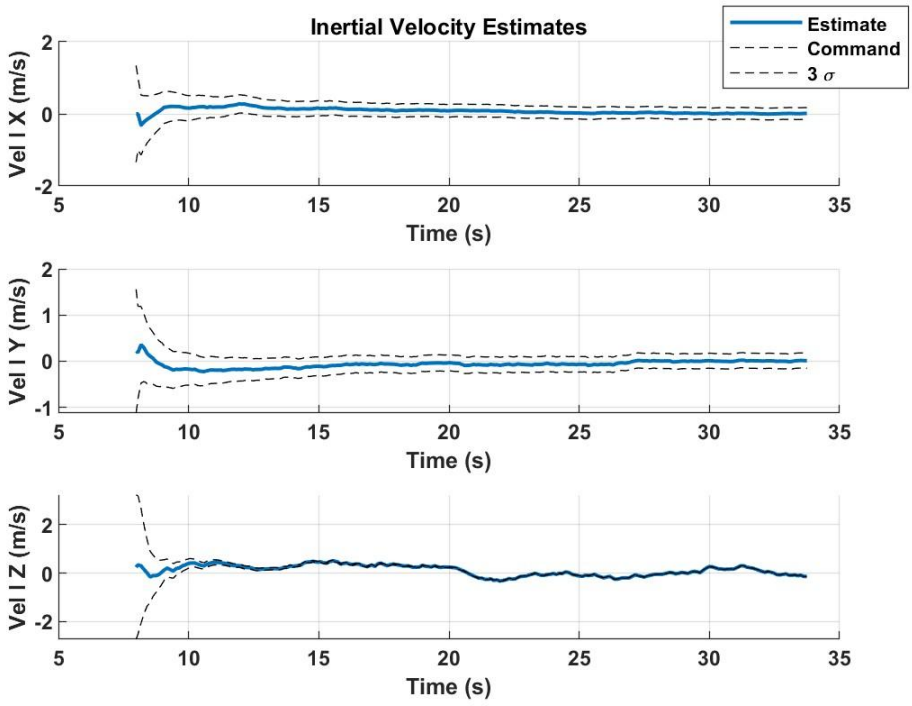
$$k_{d_{pIX}} = 10$$

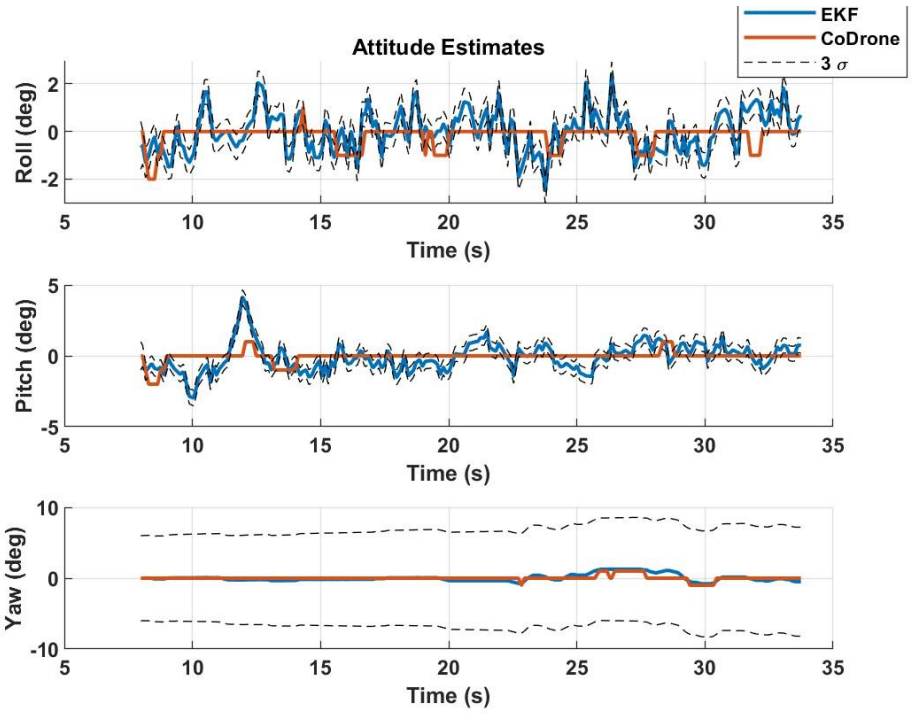
$$k_{p_{IY}} = 20$$

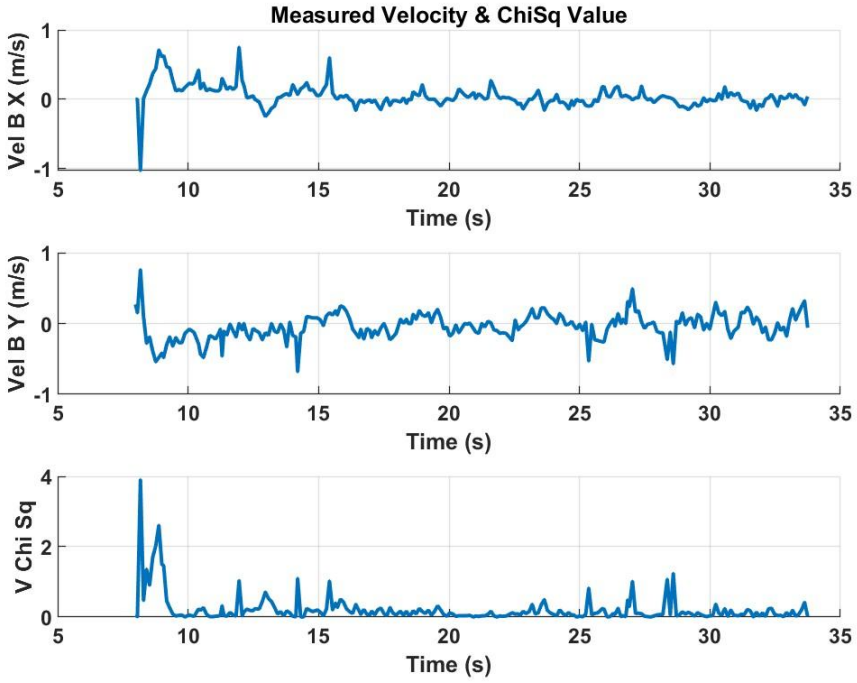
$$k_{i_{pIY}} = 4$$

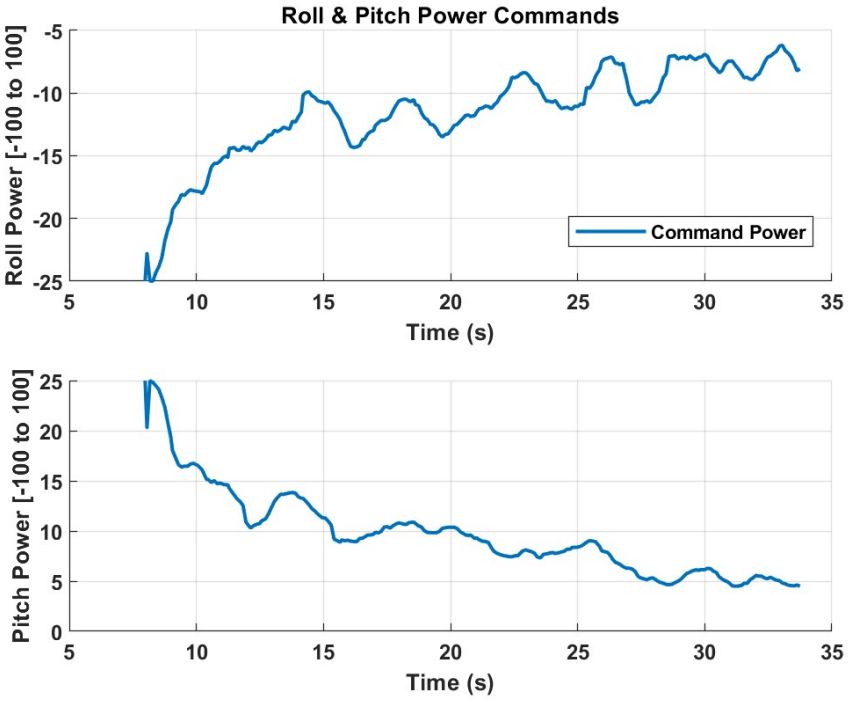
$$k_{d_{pIY}} = 10$$




Inertial Velocity Estimates
Estimate
Command
3 σ
Vel I X (m/s)
Vel I Y (m/s)
Vel I Z (m/s)
Time (s)

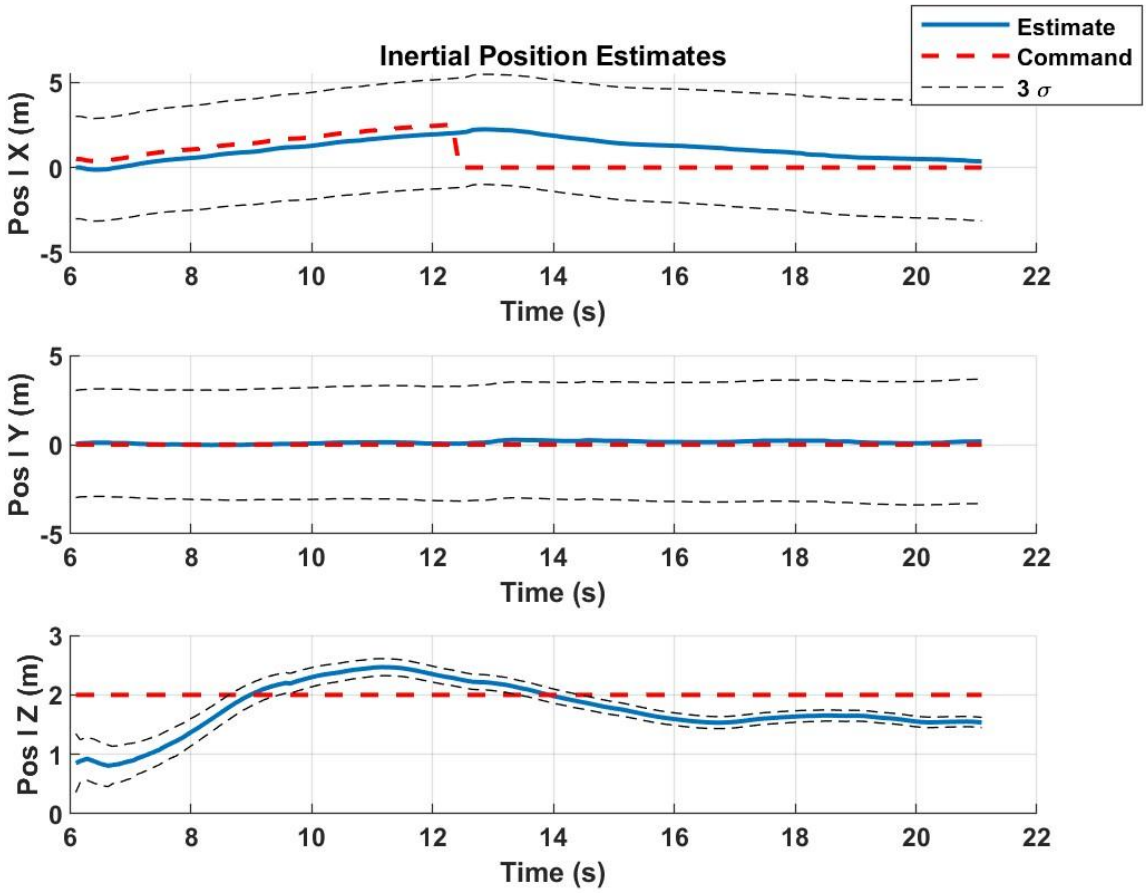

Attitude Estimates
EKF
CoDrone
3 σ
Roll (deg)
Pitch (deg)
Yaw (deg)
Time (s)


Measured Velocity & ChiSq Value
Vel B X (m/s)
Vel B Y (m/s)
V Chi Sq
Time (s)


Roll & Pitch Power Commands
Roll Power [-100 to 100]
Command Power
Pitch Power [-100 to 100]
Time (s)

For my tuning method, I started off k_p at 20 based off intuition. I tested this by commanding the CoDrone to move along the X position to 1 m. It reached 1.3 m and then swung back a bit. Then I moved k_d from 5– 15 at increments of 1, and compared the results. At 15 it was too slow and never quite reached 1m, while 5m had some small overshoot. I found that 10 worked the best for me. I then moved k_i from 0 to 5 in increments of 1 and found that 4 was consistently adequate. I would not say that the Codrone tracked its commanded position that well based on its own state estimate. It would always end up a little shorter than 1m (after measuring), however as an outside observer I would say it did well moving forward to what "looked" like 1 m. The results were all over the place which made it hard to tune, even the same gains would have big inconsistencies.

Part 4 – Trajectory Control



The CoDrone does find the wall and finds its way back to the takeoff position. It ended up being about 0.67 m away from its original position. It did a great job keeping its position during flight and following the trajectory plan. This does meet the expectation based on navigation as slight positional errors can compound a bit, resulting in the landed spot being a bit further away from the starting position. To an outside observer, it would look as though the CoDrone did not really track its position well, as it ends in a different spot than where it started. However, based off of its own state estimate, I believe it did pretty well. These would be different observations as the state estimate is not truly accurate to the real position due to sensor errors.

Final Notes

While this was a great practical experience with UAVs and its control, the finicky nature of the sensors made tuning difficult. I will say the drone battery would die pretty quickly with constant turning on and off of the drone. I had to find another mini-usb charger to have the batteries charging constantly, while also having the controller connected to the computer at the same time. The assignments seemed simple at first, but a small error in the coding or simply tuning one parameter would cause a simple task to last 2-3 hours. Whereas the simulation would be a quick restart and the ability to track multiple tuning gains, the codrone required setting it up at the same location and making sure the battery was charged. Overall, the easy tasks but “annoying” implementation made them ok to be done in within the timeline given. I will say the hardware was helpful in reinforcing the lecture material.